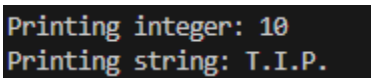
COMPUTER ENGINEERING

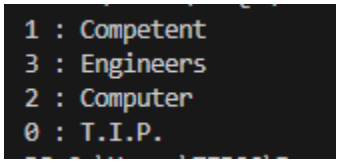
QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S2
DATE PERFORMED:	October 16, 2026	**DATE SUBMITTED:**	October 16, 2026
NAME:	Marc Jhowell Bungag Ramelissa Melith V. Limpio	**INSTRUCTOR:**	Engr. Jimlord Quejado

HOA 13.1 **Parallel Algorithms and Multithreading**

1. PROCEDURE OUTPUT

Part 1: Simple One-Threaded Example	
Screenshot	
Analysis	A single thread, t1, runs print with the arguments 10 and "T.I.P.", so the output is the same every run. t1.join() makes the main wait for t1 to finish; without it, the program could end with an error. With one thread there is no speedup, so this only shows how to create and join a thread.

Part 2: Multi-Threaded Example	
Screenshot	
Analysis	The output shows a multi-threaded program displaying a list of items with their corresponding numbers. The output includes “1: Competent Engineers,” “3: Computer,” “2: T.I.P.,” and “0: T.I.P.” (partially visible). This demonstrates how a multi-threaded program can process or display multiple pieces of information. The order of the output may vary depending on how the threads are scheduled by the operating system. This example illustrates the importance of thread execution and synchronization in ensuring that program output is displayed correctly and consistently.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Part A: Demonstrate an understanding of parallelism, concurrency, and multithreading in C++ by answering the given questions.

1. Write a definition of multithreading and its advantages/disadvantages

Limpio
Multithreading means executing multiple threads concurrently within a single program, all sharing the same memory (in C++, this is done with std::thread). It allows more efficient use of multicore CPUs and keeps applications responsive, yet it may lead to data races and deadlocks, is more difficult to debug, and introduces overhead for short tasks.

Bungag
Multithreading is a programming technique that allows a single process to execute multiple threads of tasks. A thread is the smallest unit of execution within a process. Multiple threads can share the same memory and resources while performing different tasks. Multithreading helps programs remain responsive and use system resources more efficiently, especially when handling multiple operations at the same time.

2. Rationalize the use of multithreading by providing at least 3 use-cases

Limpio
Web servers rely on it to manage numerous client requests simultaneously. Games and applications utilize it to run rendering, input, and audio independently, preventing the screen from freezing. Data processing jobs such as sorting or image processing use it to distribute work across CPU cores and complete tasks more quickly.

Bungag
Multithreading is used in web servers to handle requests from multiple users, in video and image processing to perform editing and encoding tasks faster, in games to manage audio and background activities while maintaining smooth gameplay, in file downloading and data processing to handle multiple operations efficiently, and in database systems to process multiple user requests and execute queries simultaneously while protecting shared data.

3. Differentiate between parallelism and concurrency.

Limpio
Concurrency means multiple tasks overlap in time and take turns, similar to two lines of people sharing a single espresso machine. Parallelism means multiple tasks execute at the same moment on separate cores, similar to two lines of people each having their own espresso machine.

Bungag
Concurrency is the ability of a program to manage multiple tasks whose execution overlaps in time, allowing them to make progress by switching between tasks or executing them simultaneously, while parallelism refers to the simultaneous execution of multiple tasks using different processing units, such as CPU cores, to complete work more efficiently; concurrency allows tasks to take turns using a processor and can work on a single-core processor, whereas true parallelism generally requires multiple processing units, as demonstrated when an application downloads a file while responding to user input through concurrency and when a program divides a

large calculation into independent tasks that run simultaneously on different CPU cores through parallelism.

Part B: Create C++ Code and show a solution that satisfies the given requirements below

Create a global variable of type integer.
Create an add function that will take an integer parameter and add that value to the global variable.
Use multi-threading techniques to create 3 threads; individually pass the add function to the threads.
Display the value of global variable at different combinations of using the join() per thread. Such that:

- **o Display**
- **o T1.join()**
- **o Display**
- **o T2.join()**
- **o Display**
- **o T3.join()**

Provide an analysis based on the outputs of the multi-threading exercise.

```cpp
#include <iostream>
#include <thread>
#include <mutex>
using namespace std;

int globalVar = 0;          // Global integer
mutex mtx;                  // Protects globalVar

void add(int value) {       // Adds the parameter to the global
    lock_guard<mutex> lock(mtx);
    globalVar += value;
}

void display(const string& label) {
    lock_guard<mutex> lock(mtx);
    cout << label << globalVar << endl;
}

int main() {
    thread T1(add, 10);
    thread T2(add, 20);
    thread T3(add, 30);

    display("Display: ");
    T1.join();
    display("After T1.join(): ");
    T2.join();
    display("After T2.join(): ");
    T3.join();
    display("After T3.join(): ");
    return 0;
}
```

```
Display: 0
After T1.join(): 60
After T2.join(): 60
After T3.join(): 60
```

Limpio
The global variable's displayed value depends on when join() is called. Before any join, it may show 0 or a partial total; after T1.join(), T2.join(), and T3.join(), it includes each thread's addition in turn, reaching the full total after T3. Intermediate values can vary between runs due to thread scheduling, and without a mutex or std::atomic race conditions are possible.

Bungag

The output shows how multithreading works when three threads add values to a shared global variable. The initial value is 0, and each thread adds 10, 20, and 30, respectively. The join() function makes the main program wait for each thread to finish before displaying the result. Since all three threads may finish before the first join() returns, the output already shows 60 after T1 finishes. The mutex also protects the global variable from simultaneous updates, helping prevent data conflicts.

Part C: Use multi-threading with one of the algorithms previously developed in the course; provide an analysis of the result

SuppC.cpp > main()

```cpp
#include <iostream>
#include <thread>
using namespace std;

void bubbleSort(int arr[], int start, int end) {
    for (int i = start; i < end - 1; i++) {
        for (int j = start; j < end - 1 - (i - start); j++) {
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

int main() {
    int arr[] = {9, 4, 7, 2, 8, 1, 6, 3};
    int size = 8;
    int mid = size / 2;

    cout << "Original array: ";
    for (int i = 0; i < size; i++)
        cout << arr[i] << " ";
    cout << endl;

    thread T1(bubbleSort, arr, 0, mid);
    thread T2(bubbleSort, arr, mid, size);

    T1.join();
    T2.join();

    int temp[8];
    int i = 0, j = mid, k = 0;

    while (i < mid && j < size) {
        if (arr[i] <= arr[j])
            temp[k++] = arr[i++];
        else
            temp[k++] = arr[j++];
    }

    while (i < mid)
        temp[k++] = arr[i++];

    while (j < size)
        temp[k++] = arr[j++];

```

```
42
43      while (i < mid)
44          temp[k++] = arr[i++];
45
46      while (j < size)
47          temp[k++] = arr[j++];
48
49      for (int i = 0; i < size; i++)
50          arr[i] = temp[i];
51
52      cout << "Sorted array: ";
53      for (int i = 0; i < size; i++)
54          cout << arr[i] << " ";
55      cout << endl;
56
57      return 0;
58  }
```

```
Original array: 9 4 7 2 8 1 6 3
Sorted array: 1 2 3 4 6 7 8 9
```

Limpio
The program bubble-sorts each half of the array in its own thread, then merges the two sorted halves after both threads join. The output is correct and race-free since the threads use separate index ranges. Complexity stays O(n²), and with only 8 elements, thread overhead likely outweighs any speedup, so a larger array or merge sort would show the benefit better.

Bungag
The program uses two threads to apply Bubble Sort to separate halves of the array at the same time. After both threads finish, the program merges the sorted halves to produce the final result in ascending order. This demonstrates how multithreading can divide a sorting task into smaller parts. However, for small arrays, creating threads may not improve performance because Bubble Sort has a time complexity of $O(n^2)$, and merging the results also takes additional time.

Conclusion

Limpio
This activity showed that threads run concurrently, join() controls when the main thread waits, and shared data needs synchronization. I met the requirements and understood thread behavior; my area for improvement is using mutexes and atomics consistently and testing on larger workloads.

Bungag
In conclusion, this activity helped me understand how multithreading works in C++ and how multiple threads can modify a shared global variable. I learned that using join() allows the main program to wait for threads to finish, while a mutex helps protect shared data from conflicts. Overall, multithreading can make programs more efficient by allowing tasks to run concurrently, but proper synchronization is important to ensure correct results.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

COMPUTER ENGINEERING

QUEZON CITY

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.